DevOops: What Not to Do in DevOps

A Guided Tour of Disasters, Anti-Patterns, and the SQL Scripts Nobody Tested

Speaker: Sander Stad
Event: *Conference Name Here*



Sander Stad

(he/him)

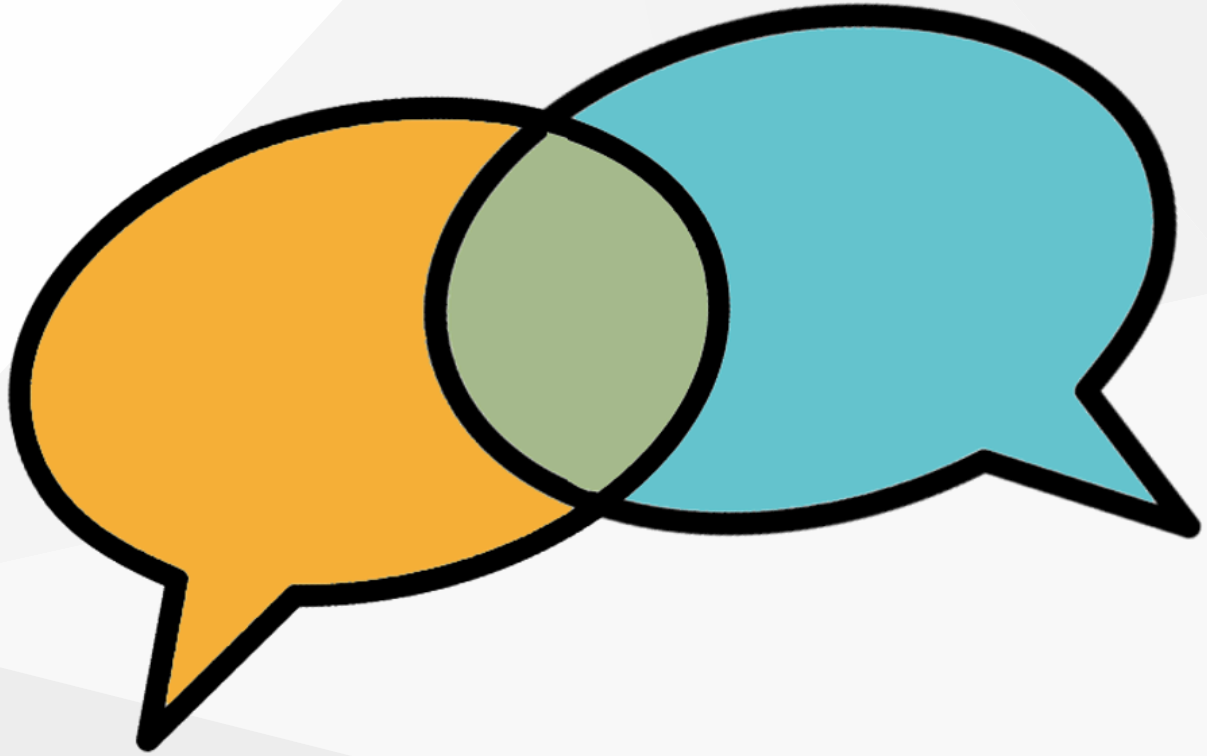
sqlstad.nl

sander@sqlstad.nl

sanderstad

Presentation Rules

- Always ask questions
- Please interrupt me if you have a question
- This is a two-way conversation, let’s learn from each other’s experiences



Been There, Shipped That 🎲

Scan to get your bingo card.

Mark a square every time you recognise your team.

Five in a row wins. Or loses. Hard to tell.

```
https://sanderstad.github.io/presentati
devoops/bingo/
```



Who Is This Talk For?

If you have ever...

- typed `DROP TABLE` on production 🔥
- merged directly to `main` on a Friday 🔥
- called a manual deployment "good enough" 🔥
- said "we don't version control our database" 🔥
- pushed a hotfix at midnight to avoid a change freeze 🔥
- said "it works on my machine" to close a prod incident 🔥

This talk is for you. You are among friends. We do not judge. (We judge a little.)

For the next 60 minutes or so ...

1. What is a DevOps Anti-Pattern? + The Rule of 10
2. Anti-Pattern #1: The Manual Deployment Hero
3. Anti-Pattern #2: The Database Nobody Touches in CI/CD
4. Anti-Pattern #3: The Silo Kingdom
5. Anti-Pattern #4: The Broken Pipeline Nobody Fixes
6. Anti-Pattern #5: Infrastructure as... a Word Document
7. Lessons + The Path Forward

“We laugh first. Then we fix things. ”

What Is a DevOps Anti-Pattern?

A DevOps anti-pattern is a behaviour that feels fast today and creates outages tomorrow.

“A good DevOps process is like a well-oiled machine.
An anti-pattern is someone poured coffee into the gears and called it agile. ”

The feedback loop is too slow to feel the pain — until 3am.

Repeat after me: **"Temporary workarounds are permanent architecture."**

The Rule of 10

Where you find the bug	Relative cost
Development	**1×**
Test / staging	**10×**
Production	**100×**

Every anti-pattern in this talk is a mechanism for pushing bugs **to the right**.

“The further right the bug travels, the more expensive the ticket. ”

⚠️ ANTI-PATTERN #1

The Manual Deployment Hero

"Don't worry, I'll deploy it myself. I know what I'm doing."

Anti-Pattern #1 — The Setup

- This person has a **47-step Word document**
- Step 23 literally says: **"pray"**
- Their desktop has `Deploy_Final_v9_REAL.docx`
- Their vacation = your change freeze
- Every deploy is a unique, artisanal, hand-crafted event

```
Ticket → Copy files → RDP → Edit config → Run SQL → Restart service → Hope
```

The Symptoms

- No repeatable process — every deploy is improvised
- "It works on my machine"
- Deployments at **11pm on a Friday** (because everyone is too scared to deploy during the day)
- One person knows everything (and they're on holiday)
- Knowledge lives in one person's head, not in version control

“Hall of shame: "The runbook is in my head." ”

The Real Cost

- MTTR is often **6× higher** with manual deployments *(DORA Research)*
- Human error contributes to **70%+** of production incidents
- Every hero model creates a single point of failure
- Vacation anxiety: team can't ship without one person present
- Manual processes don't scale — speed is also a casualty

🔥 Fast today, expensive forever.

The Human Behind It

This isn't a reckless person.

They stayed late when the release was broken. They learned the deployment by watching — because there was no documentation. They became the only one who knew the sequence **not because they hoarded knowledge, but because nobody else ever needed to learn it.**

The system made them indispensable.

Now they can't take a holiday without leaving a sticky note that says "call me if it breaks."

“We're not here to blame the hero. We're here to ask why the system needed one.

The Fix

- Automate your deployment pipeline
- Codify every step — if a human does it twice, automate it
- Validate before deploy, always
- Make rollbacks boring

```
steps:
- task: PowerShell@2
  displayName: Validate deployment
  inputs:
    script: ./scripts/Invoke-DbValidation.ps1

- task: PowerShell@2
  displayName: Deploy
  inputs:
    script: ./scripts/Invoke-DbMigration.ps1 -Environment Prod
```

If a human has to do it twice, automate it. If once… still automate it.

⚠️ ANTI-PATTERN #2

The Database Nobody Touches in CI/CD

"We version control everything... except the database. That's different."

"Databases Are Special" (They Are Not)

App path:

```
Git → Build → Test → Deploy
```

DB path:

```
Email → Hope → Manual script → Prayer → Fingers crossed
```

> ""We use Flyway / Liquibase / SSDT / nothing for migrations."
> — Nobody at this company (publicly)"

A database schema is just code that lives in a different runtime. Treat it like code.

Classic Database Anti-Patterns (Hall of Shame)

- `final_v2_ACTUALLY_FINAL_USE_THIS.sql`
- No schema versioning — nobody knows what's in prod
- `ALTER TABLE` directly on prod
- Rollback plan: "we'll figure it out"
- Migration history: one person's memory + a shared drive nobody can find

If your SQL folder looks like fan fiction, we need to talk.

The Friday Afternoon DB Migration

"It was 4:45pm on a Friday..."

- Someone ran a migration directly in production
- It locked the hottest table in the system
- API latency became "geological"
- Slack channel became a true-crime documentary
- The rollback script? In a Downloads folder. On a laptop. At a school play.

And that is why we test migrations — with production-like data volumes — before touching prod.

Database CI/CD: What Good Looks Like

- DB migration is a first-class pipeline stage
- Every migration is versioned and reviewed in a PR
- Migration tested on ephemeral/pre-prod DB with realistic data volumes
- Rollback strategy decided and tested **before** the deploy window opens

```
Commit → Build → Unit Tests → DB Validate → DB Migrate → App Deploy → Smoke Test
```

Code Example: Bad vs Good

BAD — Unversioned + Manual

```
-- random.sql
ALTER TABLE Orders
  ADD DiscountCode NVARCHAR(20);
```

Run manually via `Invoke-Sqlcmd`

GOOD — Versioned + Pipeline

```
-- migrations/V3__add_column.sql
ALTER TABLE dbo.Orders
  ADD DiscountCode NVARCHAR(20) NULL;
```

Run via

`./scripts/Invoke-DbMigration.ps1`

The Human Behind It

This is the DBA who's been burned before.

Three years ago, an untested migration locked the Orders table for 45 minutes on a Monday morning. Since then, every schema change goes through them personally. Not because they love the control — **because nobody else has stood in the blast radius.**

They approve everything manually because they've never been given an automated gate that does it for them.

“Give them the gate. The blocking stops. ”

The Fix

- Every migration is a versioned file in Git
- Schema changes reviewed in a PR like any other code
- Pipeline validates and applies — no manual scripts in prod
- Rollback is tested before the deploy window opens

Tools: Flyway · Liquibase · SSDT · Redgate SQL Change Automation

> "If your database can't be rebuilt from source control, you don't have DevOps. You have hope."

🎲 **If that's your team — mark your card.**

⚠️ ANTI-PATTERN #3

The Silo Kingdom

"That's not my problem. That's the DBA team's problem."

The Org Chart of Doom

- Dev, DBA, Ops, Security in separate castles
- Arrows between teams labeled: **"blame"**
- Four toolchains, four pipelines, four definitions of "done"
- Nobody defines "done" as "the customer can use it"

```
Dev → ticket → DBA → ticket → Ops → ticket → postmortem
```

> “Handoffs are where context goes to die. Every one adds latency and loses information.”

Symptoms of Silo Disease

- "We didn't know they were deploying today"
- "The DBAs always block us" *(give them an automated gate instead)*
- "DevOps is a Dev problem"
- App changes break DB contracts nobody discussed
- Shared responsibility means nobody's responsible

🔥 Communication debt compounds faster than technical debt.

The Data/AI Angle

- Data pipelines deployed manually by whoever owns that day
- Feature stores changed without versioned contracts
- ML models shipped outside shared release controls
- Model versioning disconnected from data versioning

“Your ML model is also software.
It has dependencies, failure modes, and needs a rollback strategy.
It needs a pipeline too.”

The Human Behind It

Nobody started this as a silo.

There was a team, a project, a deadline. Roles got carved out to move fast. The DBA team had their own sprint board. Ops had their change window. Security had their audit checklist. **Each team optimised locally and got rewarded for it.**

Three years later, there's a ticket chain that takes two weeks for a one-line config change, and nobody in any of the four teams can see the whole picture.

“The silos weren't built — they accreted. ”

The Fix: Shared Pipelines, Shared Responsibility

- Cross-functional delivery teams with shared release ownership
- Shared pipeline templates everyone contributes to
- Inner source pipeline components — DBAs are pipeline engineers too
- One definition of done that includes database, infra, and quality gates

```
extends:
  template: pipelines/templates/data-app-release.yml
  parameters:
    runDatabaseMigrations: true
    runDataQualityChecks: true
```

🎲 **If that's your team — mark your card.**

⚠️ ANTI-PATTERN #4

The Broken Pipeline Nobody Fixes

"Oh that? That's been red for three weeks. We just ignore it."

The Slowly Rotting Pipeline

- One flaky test goes red
- Nobody fixes it — "it's always done that"
- More failures pile up
- Team stops trusting CI entirely
- Eventually: "just merge it, the pipeline is always red" 💀
- A red default branch becomes accepted background noise

> “A broken window leads to more broken windows.
> A red pipeline that stays red is worse than no pipeline — it creates false confidence.”

Why People Don't Fix It

- "It's flaky, not broken" *(flaky = broken, full stop)*
- "We'll fix it after release" *(you won't — there's always another release)*
- "It's someone else's test" *(if it's in your pipeline, it's yours)*
- "We'll make a ticket for it" *(that ticket is 8 months old)*
- Fixing it requires understanding code nobody owns anymore

🔥 Unowned pipelines become organisational background noise.

The Human Behind It

The first flaky test failed on a Tuesday. Nobody knew whose code caused it. The team agreed to look into it "after the sprint."

The next sprint had a different priority. The test kept flipping — red, then green — until it became background noise. One engineer muted notifications for that check. Another set the pipeline status badge to **decorative**.

By the time three more tests failed, the team had collectively decided the pipeline was unreliable.

> "The pipeline was fine. The team had stopped reading it.
> *(Rule of 10: every bug that slipped through was caught later, at 10× or 100× the cost.)*"

The Fix: Pipeline Hygiene

- Flaky test policy: **fix or delete, never ignore** — agree before the first flake
- Explicit pipeline ownership — someone gets paged when it goes red
- Alerts when default branch goes red — this is a P1, not a backlog item
- Dashboard reviews in standup — 30 seconds to confirm green

```
if ($env:BUILD_STATUS -ne 'Succeeded') {
  Write-Error 'Pipeline unhealthy: stop merges until fixed.'
  exit 1
}
```

🎲 **If that's your team — mark your card.**

⚠️ ANTI-PATTERN #5

Infrastructure as... a Word Document

"It's all documented. Steve wrote it down in 2019."

"Just Click Here, Then Here, Then Here"

- Manual infra setup creates snowflakes — each server is subtly unique
- Nobody knows what's on that server
- "Worked fine until Steve left" — Steve was the documentation
- You can't reproduce it, audit it, or scale it

Pets vs cattle:

- **Pet server:** lovingly named, irreplaceable, terrifying to touch
- **Cattle server:** numbered, reproducible, replaceable, boring in the best way

IaC: Why It Matters for Data Teams

Your data platform **is infrastructure**:

- SQL Server / Azure SQL
- Synapse / Fabric
- Databricks workspaces
- Storage / networking / secrets / access policies

Code-define it. Pipeline-deploy it. Make "what changed?" a question with a git answer.

IaC Anti-Patterns

- One 5000-line file nobody understands or can review in a PR
- Copy-pasted templates with zero modules — 40 tenants all slightly different
- Secrets hardcoded in IaC files 💀 *(it's in Git now, forever)*
- No module reuse — changing one property means editing 12 files
- "We wrote Bicep once. It scared us." — start smaller, not stop

> “If your IaC is unreadable, it is just YAML fan fiction. ”

The Human Behind It

Steve built the server in 2019. He knew exactly what he'd installed, which firewall rules he'd opened, why port 1433 had that specific inbound exception. He wrote it all down. Eventually. In a Word document. In a folder on a shared drive nobody looks at.

When Steve left, the document was six months out of date and the server had three more things on it nobody knew about.

The new engineer provisioned a second server. **It was almost the same as the first one. Almost.**

“"Almost" is where production incidents live. ”

The Fix: Start Small with IaC

- One resource, one parameter file, one deployment pipeline
- Get one thing reproducible, then add the next

```
az deployment group create `
  --resource-group $ResourceGroup `
  --template-file ./infra/main.bicep `
  --parameters sqlServerName=$SqlServerName dbName=$DatabaseName
```

Start with the most painful resource. Get it reproducible. Then add the next.

You don't need to boil the ocean. Boil one kettle. Then boil another.

🎲 **If that's your team — mark your card.**

🎲 Been There, Shipped That

Anyone get BINGO?

The Anti-Pattern Bingo Card

B	I	N	G	O
Friday deploy	Red pipeline ignored	Manual hotfix	No DB versioning	Snowflake server
"Works on my machine"	Bus factor of one	Unreviewed SQL in prod	No rollback plan	Hidden prod config
Hero deployer	Silo blame	★ FREE	Flaky tests ignored	Runbook in Word
DB change via email	Secrets in repo	"Temporary" forever	1 giant deploy script	No pipeline owner
Direct merge to main	DevOps is a Dev problem	"Fix it after release"	On-call is one person	YAML fan fiction

Lessons Learned

- ⚠️ **Manual Deployments** → Automate everything, no heroes, boring deploys are good deploys
- ⚠️ **DB Ignored in CI/CD** → Your schema is code — version it, test it, pipeline it
- ⚠️ **Silos** → Shared pipelines, shared ownership, shared definition of done
- ⚠️ **Broken Pipelines** → Fix it or delete it — a broken gate is worse than no gate
- ⚠️ **No IaC** → If it's not in source control, it doesn't exist

The DevOps Maturity Ladder

- **Level 0:** We deploy manually and pray
- **Level 1:** We have a pipeline. It's mostly red.
- **Level 2:** Automated app deployments, manual DB changes
- **Level 3:** Full CI/CD including DB migrations and IaC
- **Level 4:** Shared pipelines, automated testing, infrastructure as code — you are dangerous

“"Where are you today? What's the ONE thing between you and the next level?" ”

The One Thing to Do Tomorrow

Pick **ONE** thing you saw today.
Go back to your team.
Fix it.

You don't need a committee or a roadmap.
You just have to stop ignoring the fire. 🔥

Thank You / Q&A

- **Speaker:** Sander Stad
- **Blog:** sqlstad.nl
- **GitHub:** sanderstad
- **Bingo:** `https://sanderstad.github.io/presentation-devoops/bingo/`

> "If the room goes quiet: 'What's the anti-pattern your team is most embarrassed about? You don't have to name names.'"